Amauri Hopewell

Southern New Hampshire University

CS-499-13445-M01 Computer Science Capstone

October 4, 2026

Milestone Four: Databases Enhancement

1. What is the artifact, and when was it created?

This is the Grazioso Salvare / Austin Animal Center work from CS-340 Client/Server Development, 2025 C-5 (September–October 2025). The original files are in the original/ folder of this zip and at https://github.com/AmauriHopewell/CS340. CRUD_Python_Module_5AH.py is the AnimalShelter class: create, read, update, and delete against MongoDB database aac, collection animals. ProjectTwoDashboard.ipynb is a Dash notebook that filters those rows by breed, age, and sex so staff can find training candidates. Read used an aggregation pipeline that grouped on animal_id to hide duplicates. The constructor took a username and password, with defaults still in the source file. This milestone is the enhancement of that CRUD module and dashboard.

2. Why this artifact, and how was it improved?

I picked this project because the original assignment was talking to MongoDB correctly and then using those queries from a client. It already had real CRUD and a real collection. The gaps were database gaps: credentials in the constructor, a caller dict passed straight into $match, uniqueness handled in the query instead of in the schema, no indexes on the fields the dashboard filters, and a Jupyter notebook as the way you run the app (Hopewell, 2026).

enhanced/animal_shelter.py keeps the AnimalShelter class. I kept MongoDB. The Module One plan said a stronger Mongo design was the next step for this code, and that is what I implemented. MONGO_USER and MONGO_PASS come from the environment. If they are missing the constructor raises ConfigurationError and does not open a client. There are no default credentials in this file. Staff may read and filter. Admin may create, update, and delete. Both roles use the Mongo login from the environment. The role argument is what limits writes. sanitize_lookup allow-lists field names and a short list of operators ($gte, $lte, $in, $eq). A $where key is rejected. read uses find. A unique index on animal_id is what stops duplicates at write time. Extra indexes sit on breed, age_upon_outcome_in_weeks, and sex_upon_outcome. counts_by_breed_and_outcome is an aggregation that counts by breed and outcome_type. The Module One sketch grouped on rescue_type. The AAC collection uses outcome_type, so that is the field I grouped on. dashboard.py is the same Grazioso filters as a normal Python process. many defaults to false, so update and delete touch one matching document unless the caller passes many=True. An empty lookup on those writes returns 0. create(None) raises QueryError. python -m pytest tests -q is 12 tests on mongomock. dashboard.py uses that same in-memory client when MONGO_USER and MONGO_PASS are unset, so the filter UI can be opened from the zip without a cluster.

The trade-off is the unique index. If the AAC dump already has duplicate animal_id values, ensure_indexes fails until those rows are cleaned. A $group on read would hide those rows. A unique index rejects them on write. dashboard.py still opens if that call fails, so a dirty import does not take down the filter UI. That is the trade-off I wrote in the Module One plan (Hopewell, 2026; OWASP Foundation, n.d.).

3. Course outcomes from Module One

The Module One plan said this enhancement would show outcome 5 first, with 4 and 3 next to it. That still holds. Outcome 5 is secrets out of source, an allow-list on queries, and staff versus admin. Outcome 4 is PyMongo, environment config, and indexes used together. Outcome 3 is unique index versus aggregation-for-dedup. Uniqueness is on the collection. Read uses find. Outcome 2 is this write-up. Outcome 1 is a GitHub branch after I take comments, the same pattern as CS-320 and CS-300. The coverage map from Module One is unchanged. The 5-2 directions said to wait for instructor comments before putting this on the ePortfolio (Southern New Hampshire University [SNHU], n.d.), so Pages still links the original CS-340 repo.

4. What I learned, and what got in the way

The first hole was the constructor defaults sitting in Git. Fail-closed from the environment is the fix. The connect log records host, port, database, and username. The password stays out of that line. Tests check that AnimalShelter(admin) with MONGO_USER and MONGO_PASS unset raises ConfigurationError.

The original read method checked that lookup was a dict. A dict can still hold $where. sanitize_lookup is the gate on field names and operators. Staff calling delete raises PermissionDenied. Empty lookup on update or delete returns 0, so a blank filter does not touch every document. create still takes a dict and inserts it. Duplicate animal_id is what the unique index catches on write.

Unique index versus dirty AAC data is the snag I would need help on in a live import. mongomock lets the tests pass without that dump. The original notebook had a map. dashboard.py has the Grazioso breed, age, and sex filters, a table, and a pie chart. I left the notebook in original/ and skipped a new JavaScript front end, which is what the Module One plan said to do until the data layer was done.

References

Hopewell, A. (2026, September 6). CS-499 Module One assignment: ePortfolio enhancement plan [Unpublished manuscript]. Southern New Hampshire University.

OWASP Foundation. (n.d.). OWASP code review guide. https://owasp.github.io/www-project-code-review-guide/

Southern New Hampshire University. (n.d.). CS 499 milestone four guidelines and rubric: Enhancement three databases.

Acknowledgment of generative AI use

I used Grok (xAI) in October 2026 to help organize this narrative against the Milestone Four prompts and to format the Word file. The code in enhanced/ is my CS-340 AnimalShelter module plus the env, allow-list, role, index, and dashboard changes listed above. I ran python -m pytest tests -q (12 passed) before keeping that count in this document.